Desafio Técnico de Cientista de Dados

Registro Municipal Integrado (RMI)

Guia completo (101): do enunciado à solução em dbt e BigQuery, com explicação de cada conceito técnico do zero.

Autor	Matheus Bianco
Stack	dbt-core + dbt-bigquery, Python, DuckDB, SQL
Fonte dos dados	gs://case_vagas/rmi (Google Cloud Storage, público)
Projeto	desafio-rmi-ds

> **Para quem é este documento**
>
> Foi escrito para quem está começando em dados. Cada termo técnico (fork, dbt, warehouse, staging, teste de dados) é explicado em linguagem simples na primeira vez que aparece. Se você já é experiente, pode pular direto para as seções 6 e 7.

Sumário

1. Introdução: o que é este desafio
2. Glossário essencial (os conceitos do zero)
3. O que o desafio pedia, parte por parte
4. Os insumos: de onde vieram os dados e quais são as tabelas
5. Análise exploratória: o que descobrimos nos dados
6. A solução: o que foi feito e como, detalhe por detalhe
7. Estratégia de testes de qualidade
8. Resultados e análise de negócio
9. Como reproduzir o projeto do zero
10. Trade-offs e próximos passos

Anexo A: arquivos entregues

1. Introdução: o que é este desafio

Este desafio é um teste técnico para a vaga de **Cientista de Dados** em um contexto de setor público: o **Registro Municipal Integrado (RMI)**, descrito como o grande repositório de dados da Prefeitura do Rio de Janeiro, que reúne informações de saúde, educação, assistência social e outros sistemas.

O objetivo não é criar um modelo de machine learning. É demonstrar uma habilidade que está no coração do trabalho de dados moderno: a **engenharia de analytics**, ou seja, pegar dados brutos e bagunçados e transformá-los em tabelas limpas, testadas, documentadas e confiáveis, prontas para alimentar relatórios e decisões. Pense nisso como a diferença entre ingredientes crus e um prato pronto para servir.

Para isso, o desafio pede o uso de uma ferramenta chamada **dbt** sobre um banco de dados analítico chamado **BigQuery**, usando dados educacionais reais (anonimizados) da rede municipal.

> **A frase mais importante do enunciado**
>
> "Profundidade importa mais que completude. Melhor ter menos models com testes excelentes e documentação clara do que muitos models superficiais." Toda a solução foi construída com esse princípio em mente.

2. Glossário essencial (os conceitos do zero)

Antes de entrar na solução, vamos definir cada termo que aparece no desafio. Não pule esta seção: ela é a base para entender todo o resto.

- **Dado bruto (raw):** O dado como ele chega da fonte, sem nenhum tratamento. Costuma ter erros, duplicatas, nomes ruins e formatos estranhos.
- **Data warehouse (armazém de dados):** Um banco de dados feito para análise (não para o dia a dia de um sistema). Guarda grandes volumes e responde perguntas analíticas rápido. Aqui o warehouse é o BigQuery.
- **BigQuery:** O data warehouse do Google, na nuvem. Você envia tabelas para lá e faz consultas em SQL sobre milhões de linhas em segundos, sem gerenciar servidores.
- **GCS (Google Cloud Storage) / bucket:** Um serviço de armazenamento de arquivos na nuvem do Google. Um 'bucket' é como uma pasta gigante na internet. Os dados do desafio estão em um bucket público chamado case_vagas.
- **Parquet:** Um formato de arquivo para dados em tabela, otimizado para análise. Diferente de um CSV, ele guarda os dados por coluna e já sabe o tipo de cada campo, o que o torna muito mais rápido e leve.
- **SQL:** A linguagem usada para consultar e transformar dados em bancos. Comandos como SELECT, WHERE, JOIN e GROUP BY são SQL.
- **dbt (data build tool):** A ferramenta central do desafio. Com ela, você escreve as transformações dos dados em arquivos SQL organizados, e o dbt cuida de rodar tudo na ordem certa, testar e documentar. É como um 'framework' para organizar o SQL de um projeto de dados de forma profissional.
- **Model (model do dbt):** Cada arquivo .sql no dbt é um 'model'. Ele contém um SELECT que define uma nova tabela ou view. Um model pode usar outro como fonte.

- **Source (fonte):** A declaração, no dbt, de onde estão os dados brutos (as tabelas que você carregou no warehouse). É o ponto de partida do fluxo.
- **View x Table:** Uma 'table' (tabela) guarda os dados fisicamente. Uma 'view' é só uma consulta salva que roda na hora em que é usada (não ocupa espaço com dados). O dbt chama a escolha entre elas de 'materialização'.
- **Materialização:** Como o dbt grava o resultado de um model: como view, table, ephemeral (só uma consulta reaproveitada, sem criar objeto) ou incremental (atualiza só o que mudou).
- **Camadas: staging, intermediate, marts:** Uma organização em níveis. Staging limpa cada tabela bruta. Intermediate combina tabelas e aplica regras de negócio. Marts entrega a tabela final que responde a uma pergunta. É uma linha de montagem.
- **Teste de dados:** Uma verificação automática da qualidade. Ex: 'a coluna id nunca pode ser nula' ou 'a nota tem que estar entre 0 e 10'. Se a regra falhar, o teste acusa.
- **Lineage (linhagem):** O mapa que mostra de onde cada tabela veio e para onde vai. Ajuda a entender dependências e o impacto de uma mudança.
- **Git e repositório:** Git é o sistema que controla versões do código (guarda o histórico de mudanças). Um 'repositório' (ou repo) é a pasta do projeto versionada por ele, normalmente hospedada em sites como GitHub.
- **Fork:** Fazer um 'fork' é criar uma cópia sua de um repositório de outra pessoa, na sua própria conta. Você trabalha na sua cópia sem mexer no original. O desafio pede um fork para você colocar sua solução.
- **Commit e Pull Request (PR):** Um 'commit' é um ponto salvo no histórico do Git, com uma mensagem do que mudou. Um 'Pull Request' é um pedido para juntar suas mudanças ao projeto principal, normalmente revisado por outra pessoa.
- **CI (Integração Contínua):** Um robô que roda testes automaticamente toda vez que você envia código, garantindo que nada quebrou. Aqui usamos o GitHub Actions para rodar o dbt.
- **Anonimização:** Processo de remover ou embaralhar dados pessoais (nomes, CPF, endereço) para proteger a privacidade. Nos dados do desafio, IDs viraram códigos sintéticos e números foram aleatorizados.
- **Hash:** Uma função que transforma um valor em um código de tamanho fixo, de um jeito que não dá para voltar ao original. Usado aqui para anonimizar IDs e bairros.
- **Grão (granularidade):** O que uma linha da tabela representa. Ex: 'uma linha por aluno' ou 'uma linha por aluno/dia'. Definir o grão certo evita contagens erradas.

3. O que o desafio pedia, parte por parte

O enunciado divide o trabalho em três partes. Abaixo, cada uma em linguagem simples.

Parte 1: Projeto dbt e Modelagem

Construir um projeto dbt do zero que pegue os dados brutos e os organize em camadas. Em detalhe, pedia:

- Configurar o projeto com as 'sources' apontando para os dados e uma estrutura de pastas organizada.
- Criar a camada de **staging** para pelo menos 4 tabelas, com limpeza, padronização de nomes, tipagem e tratamento de nulos.
- Criar pelo menos 1 model **intermediate** que combine várias fontes (ex: alunos + matrículas + frequência).
- Construir pelo menos 1 **mart** que responda a uma pergunta real de gestores públicos.

Parte 2: Testes e Qualidade de Dados (a mais importante)

O enunciado diz explicitamente que esta é a parte mais importante. Pedia:

- Aplicar testes genéricos prontos do dbt (unique, not_null, accepted_values, relationships), com critério: testar o que importa, e explicar por quê.
- Criar pelo menos 2 testes customizados de **regra de negócio** (ex: nenhuma frequência pode ter data anterior à matrícula).

Parte 3: Documentação e Análise

- Produzir documentação (README) que permita a um novo membro entender e manter o projeto: setup, lineage, decisões de arquitetura e trade-offs.
- Opcionalmente (diferencial), um notebook de análise exploratória em Python.

Diferenciais sugeridos

Uso de pacotes dbt (dbt_utils, dbt_expectations, codegen), configuração de 'freshness' nas sources, uso de tags e exposures, data contracts, análise exploratória e CI com GitHub Actions. **Todos foram implementados nesta solução.**

Como o desafio é avaliado

Critério	O que observam
Modelagem e arquitetura dbt	Organização das camadas, nomes, materializações.
Estratégia de testes e qualidade	Cobertura intencional e testes de regra de negócio.
SQL e lógica analítica	Clareza e correção das transformações.
Documentação e comunicação	README, lineage e explicação das decisões.

4. Os insumos: de onde vieram os dados e quais são as tabelas

De onde foram baixados

Os dados estão em um **bucket público do Google Cloud Storage**. O enunciado aponta o endereço do console:

```
https://console.cloud.google.com/storage/browser/case_vagas/rmi
```

Investigando o bucket, descobri dois detalhes práticos importantes:

- Os arquivos foram salvos **sem a extensão .parquet** (o objeto se chama apenas 'aluno', 'escola', etc.), então baixar por 'aluno.parquet' dá erro 404.
- O bucket também contém um arquivo **schema.yml** com a descrição oficial das colunas, o que ajudou a confirmar tipos e significados.

O download direto de cada tabela pode ser feito por URL pública, por exemplo:

```
https://storage.googleapis.com/case_vagas/rmi/aluno
https://storage.googleapis.com/case_vagas/rmi/escola
https://storage.googleapis.com/case_vagas/rmi/turma
https://storage.googleapis.com/case_vagas/rmi/avaliacao
https://storage.googleapis.com/case_vagas/rmi/frequencia
```

> **Tabelas disponibilizadas para você ver**
>
> Exportei amostras legíveis das 5 tabelas (200 linhas cada) para a pasta `docs/amostras/`, em dois formatos:
>
> 1) Um arquivo por tabela em CSV (ex: `aluno_amostra.csv`).
>
> 2) Um único Excel, `RMI_tabelas_amostras.xlsx`, com uma aba por tabela, mais uma aba de volumetria e uma de dicionário de dados.
>
> Nessas amostras, o id_aluno (que no original é um código binário) aparece convertido para texto hexadecimal, para ficar legível.

As cinco tabelas e a auxiliar

Tabela	O que representa	Grão (uma linha é...)	Linhas
aluno	Cadastro de alunos	um aluno	97.044
escola	Cadastro de escolas	uma escola	1.558
turma	Na prática, a matrícula aluno-turma	um aluno/turma	97.044
avaliacao	Notas por bimestre (formato largo)	um aluno/turma/bimestre	221.687
frequencia	Frequência por período e disciplina	aluno/turma/escola/período/disciplina	2.727.684

Há ainda uma tabela auxiliar pública no BigQuery, `datario.dados_mestres.bairro`, com o cadastro de bairros do Rio. Como os bairros nos dados estão em hash (não dá para cruzar diretamente), ela não foi necessária para os marts construídos.

Dicionário de dados (colunas principais)

Tabela.coluna	Tipo	Descrição
aluno.id_aluno	BYTES	ID do aluno (código binário; vira hexadecimal no projeto).
aluno.id_turma	INT64	Turma de matrícula.
aluno.faixa_etaria	STRING	0-5, 6-10, 11-14, 15-17, 18+.
aluno.bairro	INT64	Hash do bairro (tem alguns nulos).
escola.id_escola	INT64	ID da escola.
escola.bairro	INT64	Hash do bairro da escola.
turma.ano	INT64	Ano letivo anonimizado para 2000.
turma.id_turma / id_aluno	INT64 / BYTES	Liga aluno à turma (matrícula).
avaliacao.bimestre	STRING	1 a 4.
avaliacao.disciplina_1..4	FLOAT64	Notas: Português, Ciências, Inglês, Matemática.
avaliacao.frequencia	FLOAT64	Frequência do bimestre (0-100; ~99% preenchida).
frequencia.data_inicio/fim	STRING	Período de apuração (ano 2000).
frequencia.disciplina	STRING	disciplina_1..4.
frequencia.frequencia	FLOAT64	Percentual de presença (0 a 100).

5. Análise exploratória: o que descobrimos nos dados

Antes de modelar, baixei as tabelas e investiguei com uma ferramenta chamada **DuckDB** (um banco analítico que roda no próprio computador, ótimo para explorar arquivos Parquet rapidamente). Essa etapa, a **análise exploratória de dados (EDA)**, revelou vários problemas de qualidade que guiaram todas as decisões seguintes. Entender os dados antes de transformá-los é o que separa um trabalho amador de um profissional.

1. O ID do aluno é binário, não texto

O campo id_aluno vem como BYTES (um hash de 17 bytes), e não como texto. Se usado cru, fica ilegível. **Decisão:** converter para texto hexadecimal logo no staging, criando uma chave estável e legível.

2. A tabela 'turma' é, na verdade, matrícula

Apesar do nome e da descrição do enunciado (que fala em série e turno), a tabela só tem ano, id_turma e id_aluno, com uma linha por aluno. **Decisão:** tratá-la como tabela de matrículas e não inventar colunas que não existem.

3. A tabela de avaliação está em formato 'largo'

Cada disciplina é uma coluna (disciplina_1 a 4), em vez de uma linha por disciplina. **Decisão:** 'girar' a tabela para o formato 'longo' (uma linha por aluno/bimestre/disciplina) usando UNPIVOT.

4. A disciplina de Inglês está 100% vazia

Na avaliação, a coluna disciplina_3 (Inglês) não tem nenhuma nota preenchida: zero de 221.687. **Decisão:** manter a estrutura e um teste que torna essa ausência visível, em vez de escondê-la.

5. Existem duas frequências, e só uma tem sinal

A tabela `frequencia` (por período) fica sempre entre 97% e 100%, sem variação, então mediria absenteísmo zero. Mas há um segundo campo, `avaliacao.frequencia` (frequência geral por bimestre), que varia de 0 a 100% (média 89%). **Decisão:** medir o absenteísmo pela `avaliacao.frequencia`, que é a fonte com sinal real, e manter a outra apenas como contraste. Esse foi o achado que mais influenciou a análise.

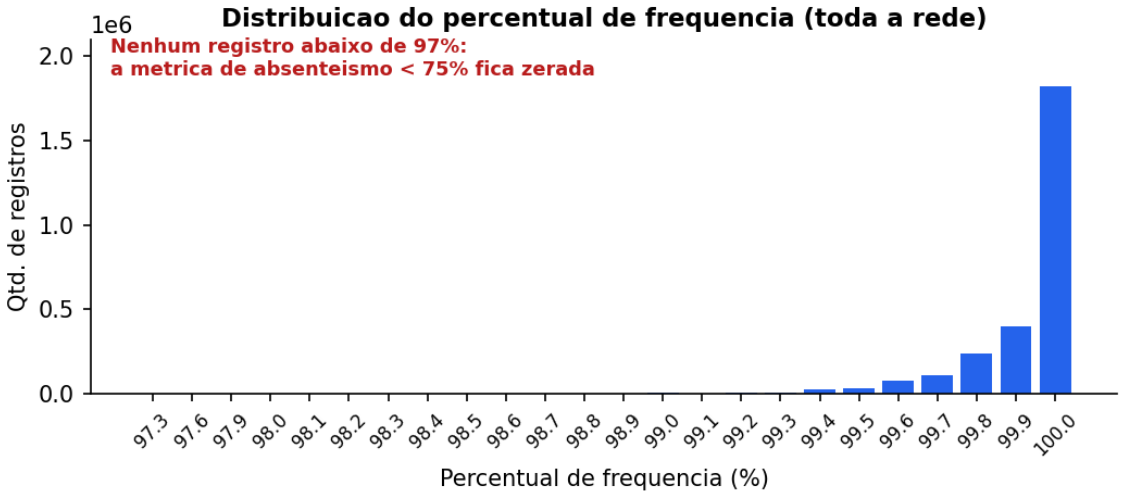
6. Chaves que não conciliam

Cerca de 12% das linhas de frequência apontam para uma turma que não existe na matrícula. Já o id_aluno bate 100% entre todas as tabelas. **Decisão:** fazer os cruzamentos pelo id_aluno (confiável) e tratar a relação por turma como apenas um aviso, não um erro que quebra o pipeline.

7. Linhas duplicadas

Há cerca de 2.600 duplicatas exatas na frequência e 34 na avaliação. **Decisão:** remover duplicatas no staging com uma técnica de numeração de linhas (row_number).

A fonte óbvia (tabela frequencia) não varia



Fonte: gs://case_vagas/rmi/frequencia (2,7M registros).

6. A solução: o que foi feito e como, detalhe por detalhe

A solução é um projeto dbt chamado `desafio-rmi-ds`, organizado nas três camadas clássicas. O diagrama abaixo mostra o 'lineage', o caminho que os dados percorrem da esquerda (bruto) para a direita (pronto para análise).

Mapa do projeto (lineage)

Fontes (BigQuery raw)	Staging (views)	Intermediate	Marts (tables)
aluno	stg...__alunos	int...__aluno_frequencia	mart...__absenteismo_regiao
escola	stg...__escolas	int...__aluno_desempenho	mart...__desempenho_por_turma
turma	stg...__matriculas		
avaliacao	stg...__avaliacoes		
frequencia	stg...__frequencia		

A regra de nomes segue o padrão `camada_dominio__entidade`. O domínio aqui é 'educacao'. Assim, quando o projeto crescer para saúde ou assistência social, os nomes continuam organizados.

Passo 0: trazer os dados para o BigQuery (ingestão)

Antes do dbt, os Parquet precisam estar no warehouse. Escrevi um script Python (`scripts/load_to_bigquery.py`) que carrega cada arquivo direto do bucket para um conjunto de dados 'raw' no BigQuery. O próprio BigQuery lê arquivos Parquet direto do GCS, então nem é preciso baixar no computador.

Camada 1: Staging (limpar cada tabela)

Cada tabela bruta ganha um model de staging (materializado como **view**, porque é leve e sempre reflete a fonte). O que cada um faz:

- **stg_educacao__alunos:** converte o id_aluno de binário para hexadecimal, renomeia e padroniza a faixa etária.
- **stg_educacao__escolas:** padroniza os nomes das colunas de escola.
- **stg_educacao__matriculas:** renomeia a tabela 'turma' para o que ela realmente é (matrícula).
- **stg_educacao__frequencia:** converte datas de texto para data de verdade, traduz os códigos de disciplina (disciplina_1 vira 'Português') e remove duplicatas.
- **stg_educacao__avaliacoes:** gira o formato largo para longo (UNPIVOT) e remove duplicatas.

Exemplo real do staging de alunos, com a conversão do ID e a padronização:

```
        select
            to_hex(id_aluno)                as aluno_id,    -- binario -> hexadecimal
            id_turma                        as turma_id,
            nullif(trim(faixa_etaria), '')  as faixa_etaria,
            bairro                          as bairro_id
        from {{ source('rmi_educacao', 'aluno') }}
```

O trecho `{{ source(...) }}` é a forma do dbt de dizer 'leia a tabela bruta chamada aluno'. Isso cria automaticamente o lineage.

Camada 2: Intermediate (combinar e aplicar regras)

Aqui as tabelas limpas se juntam e nascem as métricas. São materializados como **ephemeral** (viram apenas um trecho de consulta reaproveitado, sem criar tabela física), o que mantém o warehouse limpo.

- **int_educacao__aluno_frequencia:** junta aluno + frequência pelo id_aluno (a chave confiável) e calcula, por aluno, a frequência média, mínima e máxima, além de marcar quem tem absenteísmo crônico. A escola do aluno é definida pela 'escola predominante' (onde ele tem mais registros).
- **int_educacao__aluno_desempenho:** a partir das notas, calcula a média geral do aluno e a média por disciplina.

Camada 3: Marts (responder perguntas de negócio)

São as tabelas finais, materializadas como **table** para consulta rápida em dashboards. Cada uma responde a uma pergunta de gestor:

- **mart_educacao__absenteismo_regiao:** por escola e região, qual a taxa de alunos com frequência média abaixo de 75%? (Resposta nos dados: zero, pelo achado 5. O mart também expõe a distribuição, para funcionar quando houver dados reais.)
- **mart_educacao__desempenho_por_turma:** como o tamanho da turma se relaciona com o desempenho médio? (O enunciado sugeria série/turno, que não existem; adaptei para o tamanho da turma, que dá para calcular.)

Peças de apoio

- **Seed:** uma pequena tabela de apoio (CSV) com o de/para dos códigos de disciplina.
- **Macro:** um pedaço de SQL reutilizável (nome_disciplina) que traduz os códigos, evitando repetição.
- **Exposure:** uma declaração de que um painel consome os marts, para rastrear o impacto de mudanças.
- **Pacotes:** dbt_utils, dbt_expectations e codegen, bibliotecas prontas da comunidade.

7. Estratégia de testes de qualidade

Esta é a parte mais valorizada. A ideia é simples: um teste é uma afirmação sobre os dados que o dbt verifica sozinho. Se a afirmação for falsa, o teste falha e avisa. Isso impede que dados errados cheguem aos relatórios. A cobertura foi **intencional**: cada teste responde a uma pergunta de confiabilidade.

Testes genéricos (prontos)

Teste	O que garante	Exemplo de uso
unique	Não há valores repetidos	aluno_id na dimensão de alunos
not_null	Não há valores vazios	chaves e colunas obrigatórias
accepted_values	Só valores de uma lista	bimestre só pode ser 1,2,3,4
relationships	Toda chave existe na outra tabela	aluno da frequência existe em alunos
accepted_range	Valor dentro de um intervalo	nota entre 0 e 10; frequência 0 a 100

Um detalhe de critério: a relação da frequência com a matrícula pela turma foi marcada como **apenas aviso** (warn), de propósito, porque sabemos que 12% não conciliam. Em vez de deixar o pipeline quebrar por um problema conhecido da origem, o teste registra o aviso e seguimos cruzando pelo id_aluno.

Teste genérico customizado

Criei um teste próprio, `proporcao_nulos_abaixo_de`, que permite uma fração conhecida de nulos e só avisa quando a qualidade piora. Diferente do not_null (que exige zero nulos), ele monitora o bairro do aluno, que tem ~0,07% de nulos, sem travar o projeto.

Testes de regra de negócio (singular)

Além dos genéricos, escrevi testes em SQL que validam regras do mundo da educação. Cada um retorna linhas apenas quando a regra é violada (ou seja, passar é não retornar nada):

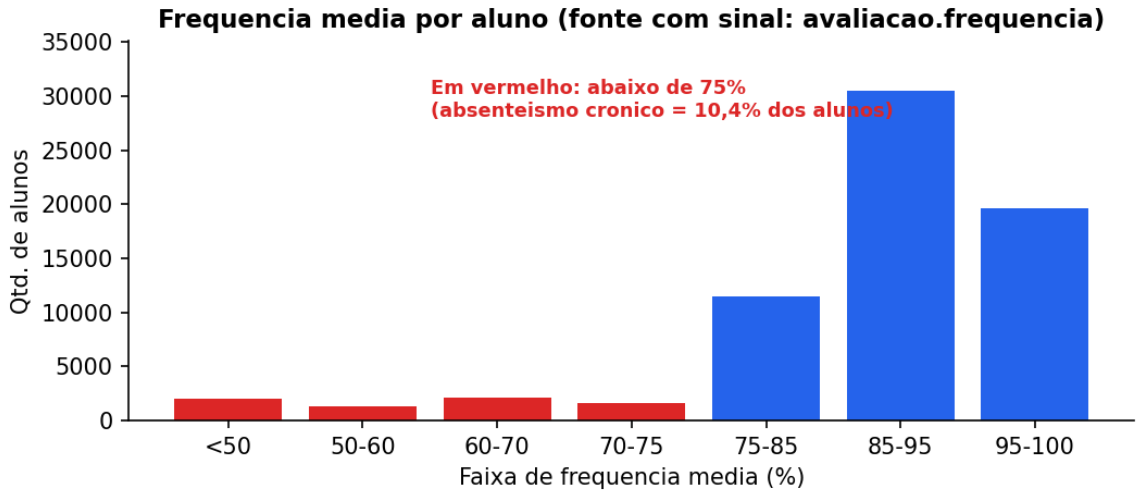
- **Datas coerentes:** nenhum período de frequência pode começar depois de terminar (data_inicio > data_fim).
- **Grão sem duplicatas:** depois da limpeza, cada combinação aluno/turma/escola/período/disciplina na frequência deve ser única.
- **Nota exige matrícula:** nenhum aluno pode ter nota lançada sem possuir matrícula correspondente.

> **Por que isso importa**
>
> Testes de regra de negócio são o que diferencia 'dados que existem' de 'dados em que dá para confiar'. Eles codificam o conhecimento do domínio (educação) dentro do pipeline, de forma que qualquer carga futura seja verificada automaticamente.

8. Resultados e análise de negócio

Absenteísmo por região

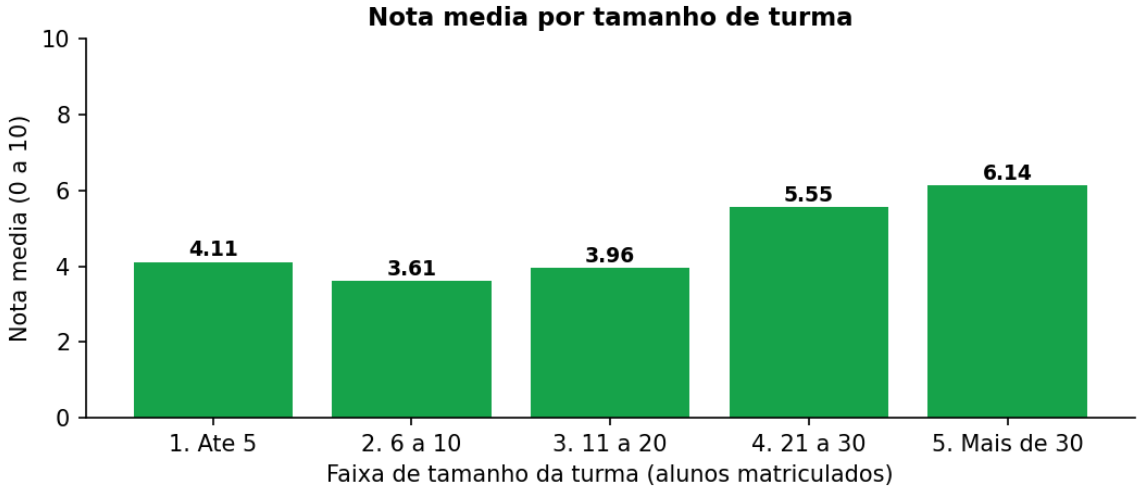
A fonte óbvia para absenteísmo seria a tabela `frequencia`, mas ela não tem variação (achado 5), o que daria taxa zero em toda a rede. Ao explorar os dados, encontrei o sinal em outro campo, a `avaliacao.frequencia` (frequência geral por bimestre), e construí o mart sobre ela. O resultado passa a ser acionável: cerca de **10,4% dos alunos** têm frequência média abaixo de 75%, e a taxa varia de **0% a 33% entre as escolas** (478 escolas com 20 ou mais alunos, média de 10,5%). Escolher a fonte certa, e documentar por que a tabela `frequencia` foi descartada, é o coração do raciocínio analítico que o desafio pede.



Fonte: avaliacao.frequencia (media por aluno, 68.735 alunos com avaliacao).

Desempenho por tamanho de turma

Aqui aparece um sinal real e interessante: turmas maiores têm nota média mais alta. Turmas de até 5 alunos têm média ~4,1, enquanto turmas com mais de 30 alunos têm média ~6,1. Vale lembrar que, por serem dados aleatorizados, isso não deve ser lido como conclusão pedagógica real; provavelmente reflete como as notas foram distribuídas. O importante é que o pipeline consegue cruzar tamanho de turma com desempenho de ponta a ponta.



Fonte: tabelas avaliacao + turma (matricula). Notas 0-10.

9. Como reproduzir o projeto do zero

Passo a passo, assumindo que você nunca rodou dbt. Você vai precisar de Python, uma conta Google (o BigQuery tem um modo gratuito, o 'Sandbox') e o utilitário gcloud para autenticar.

1) Preparar o ambiente Python

```
python -m venv .venv
.venv\Scripts\activate          # Windows
pip install -r requirements.txt
```

2) Autenticar no Google Cloud

```
gcloud auth application-default login
$env:GCP_PROJECT = "seu-projeto-sandbox"   # PowerShell
```

3) Carregar os dados do bucket para o BigQuery

```
python scripts/load_to_bigquery.py --dataset rmi_raw --location US
```

4) Configurar o acesso do dbt e rodar

```
copy profiles.yml.example %USERPROFILE%\.dbt\profiles.yml
dbt deps        # baixa os pacotes
dbt build       # cria e TESTA tudo de uma vez
dbt docs generate && dbt docs serve   # documentacao visual com o lineage
```

> **Dica para iniciantes**
>
> O comando `dbt build` faz tudo em sequência e na ordem certa: cria as views e tabelas (dbt run), carrega os seeds e roda todos os testes (dbt test). Se algo estiver errado nos dados, ele para e te mostra qual teste falhou.

10. Trade-offs e próximos passos

Nenhuma solução é perfeita. Decisões que tomei e o que faria com mais tempo:

- Usei intermediate como ephemeral (warehouse limpo), abrindo mão de inspecionar a tabela intermediária isoladamente.
- A escola do aluno foi definida por heurística (escola predominante), já que os dados não trazem um vínculo oficial.
- Com mais tempo: materializar a frequência como incremental e particionada por mês (é a maior tabela, 2,7M linhas), adicionar data contracts, snapshots para histórico e mais testes de distribuição.
- Investigar com a equipe de origem por que 12% das turmas da frequência não batem com a matrícula.

Anexo A: arquivos entregues

Arquivo / pasta	O que é
desafio-rmi-ds/	O projeto dbt completo.
README.md	Documentação técnica para manutenção.

Arquivo / pasta	O que é
models/staging, intermediate, marts	Os models SQL em cada camada.
tests/	Os testes de regra de negócio.
macros/	Macro de disciplina e teste customizado.
seeds/	Tabela de apoio (de/para de disciplina).
scripts/	Ingestão dos dados para o BigQuery.
notebooks/eda.ipynb	Análise exploratória em Python.
.github/workflows/ci.yml	Robô de testes (CI).
docs/amostras/	Amostras das tabelas em CSV e Excel (para você ver os dados).
docs/Desafio-RMI-Guia-101.pdf	Este documento.

Fim do guia. Qualquer conceito que ainda tenha ficado confuso pode ser aprofundado na documentação oficial do dbt (docs.getdbt.com) e do BigQuery.